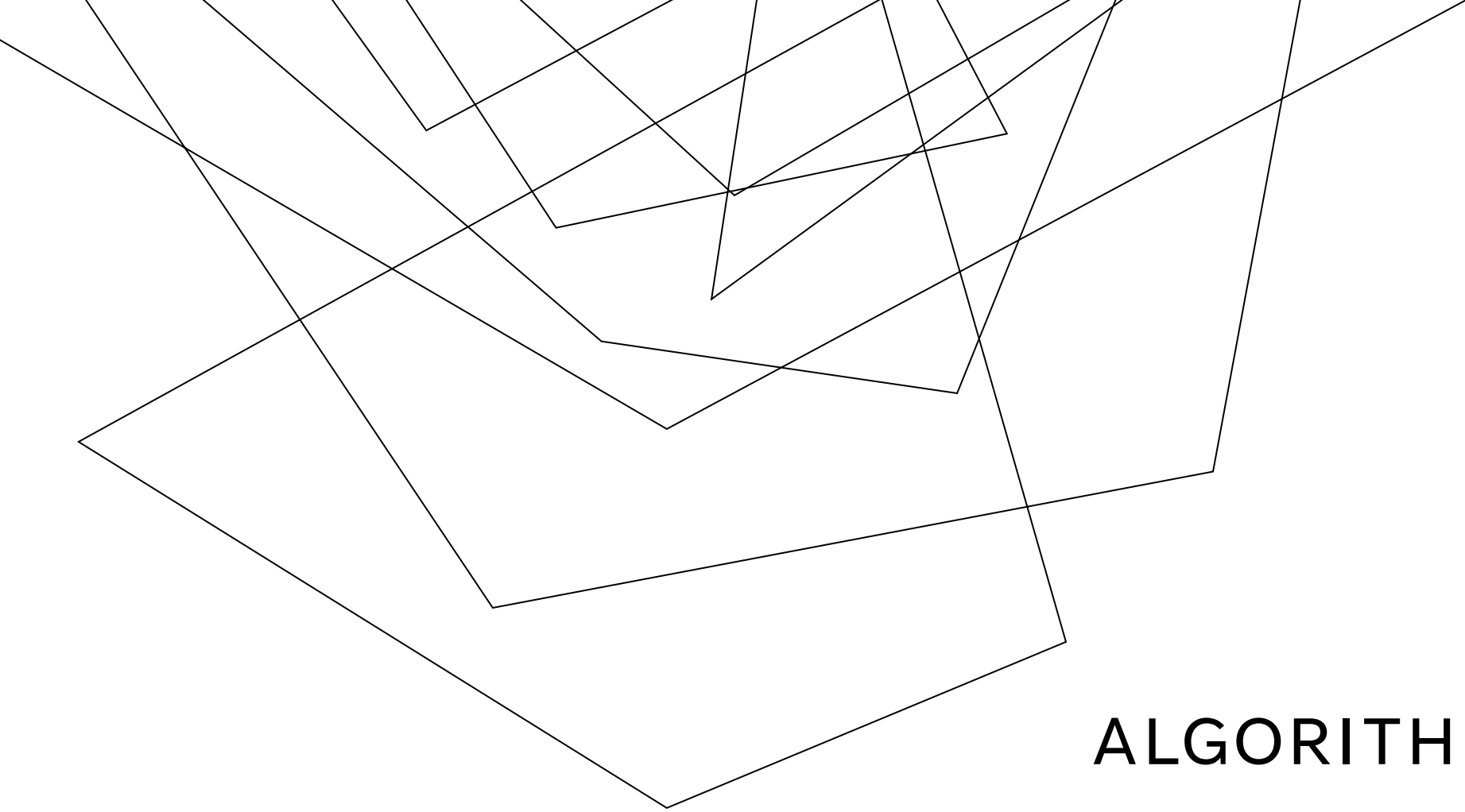


A series of thin, black, overlapping lines forming various geometric shapes like triangles and polygons, creating a complex, abstract pattern in the upper left portion of the slide.

ALGORITHMS

6. SORTING (3)





지난 시간 복습

SORTING (2)

1. Merge sort

2. Quick sort





목차

목차

1. Heap (data structure)

2. Heapsort

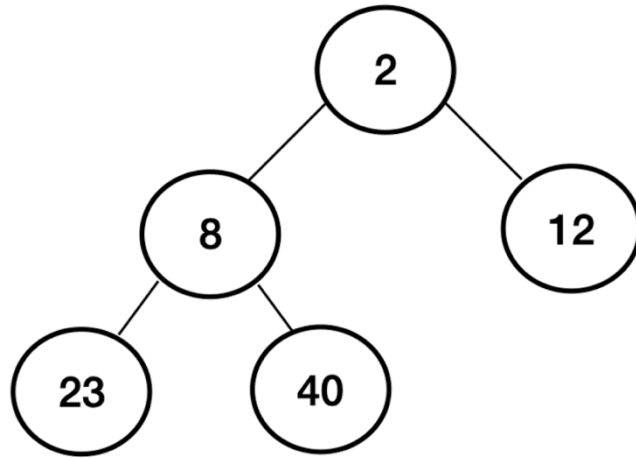




HEAP

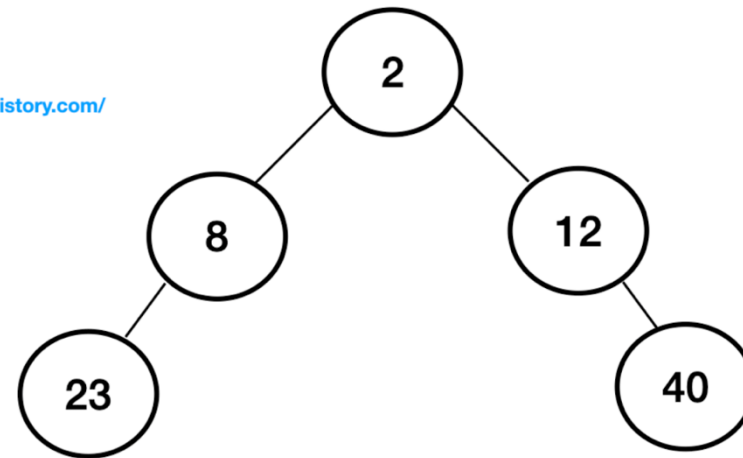
HEAP

- 완전 이진 트리



완전 이진 트리(O)

<https://heytech.tistory.com/>



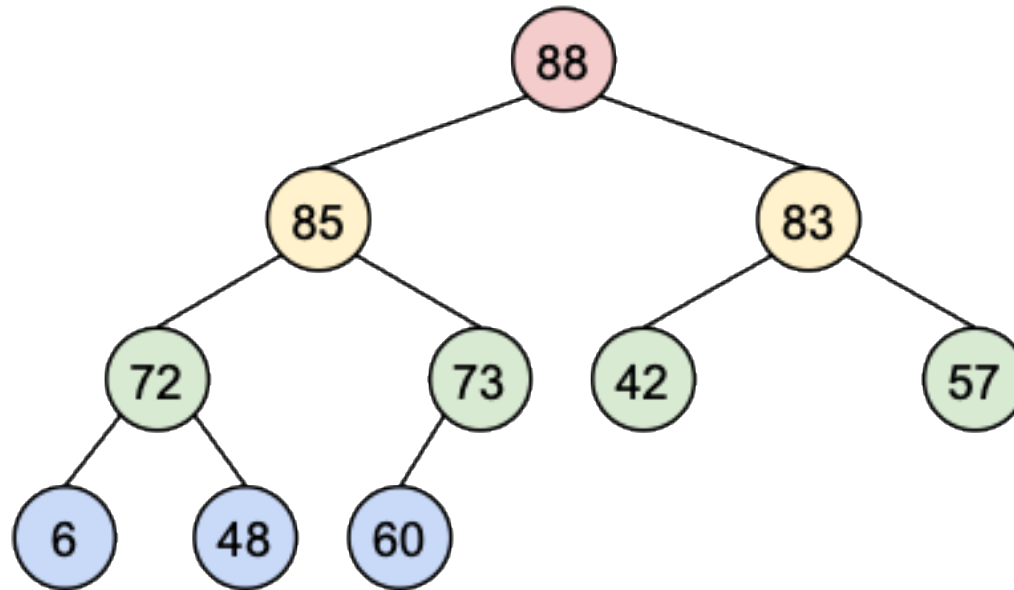
완전 이진 트리(X)

- 각 노드의 값 $\geq$ 자식 노드의 값 (Heap property)



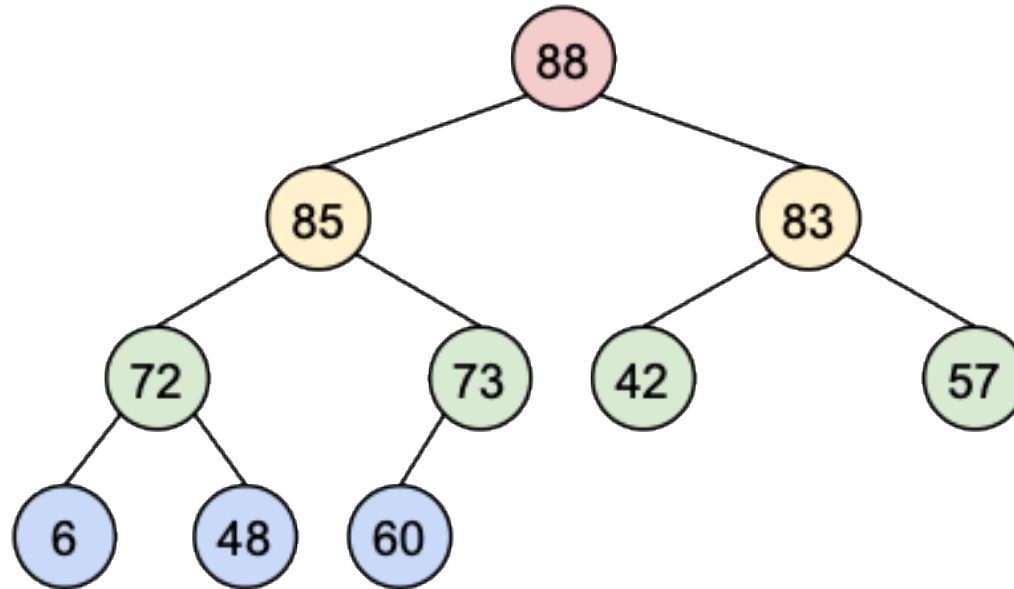
HEAP

- Heap의 예시

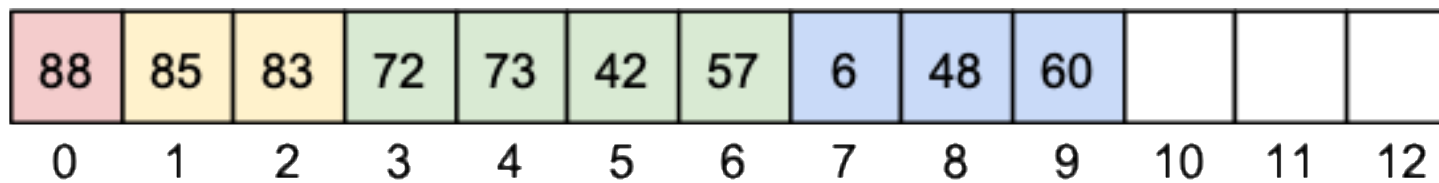


HEAP

- 완전 이진 트리는 배열로 표현 가능

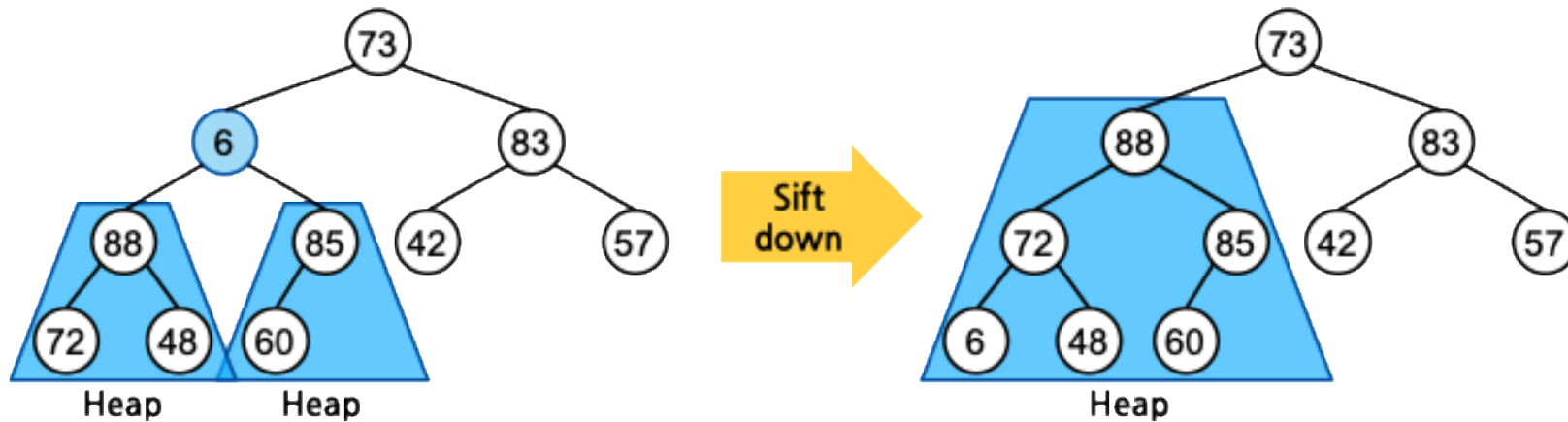


- $A[k]$ 의 자식 노드는 $A[2k+1]$, $A[2k+2]$



SIFT DOWN

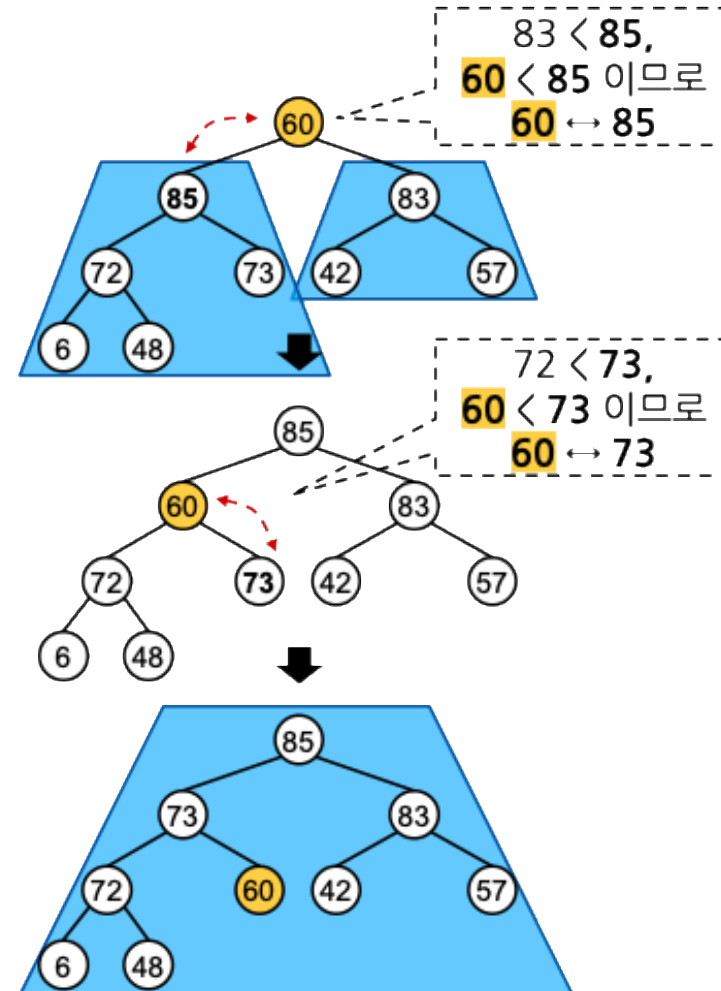
- 주어진 노드가 Heap property를 만족하지 않을 때, 해당 노드를 아래로 내려보내는 Operation (Heap property가 만족하도록)
- 자식 노드들은 각각 Heap property 만족하고 있음을 가정



SIFT DOWN

- Left child, right child 중 더 큰 값과 교환 (Swap)

```
def siftdown(i, arr, n):  
    c = 2*i+1 # c: left child  
    while c < n:  
        if c+1 < n and arr[c] < arr[c+1]:  
            c = c+1 # c: right child  
        if arr[i] > arr[c]:  
            break  
        arr[i], arr[c] = arr[c], arr[i]  
        i = c  
        c = 2*i+1
```

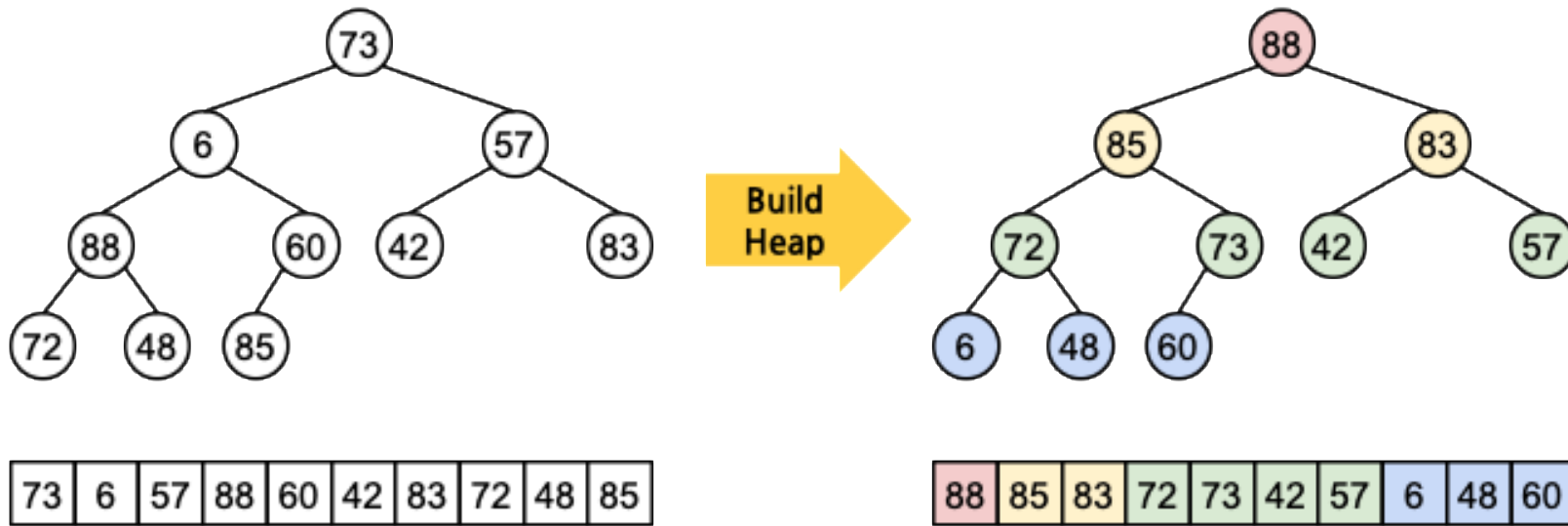


- 점근적 시간 복잡도 :



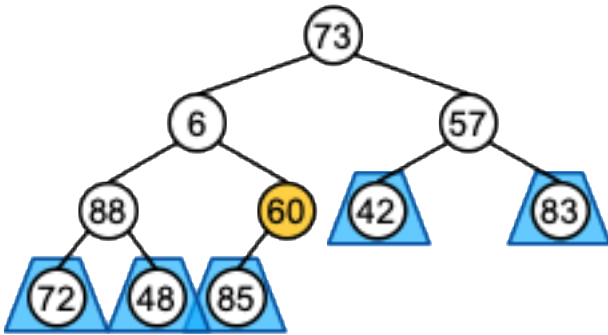
BUILD-HEAP

- 주어진 완전 이진 트리 (배열) 내 원소들을 Heap property가 만족하도록 재배열하는 Operation



BUILD-HEAP

- (자식을 가진) 부모 노드 맨 뒤에서부터 sift down 연산을 반복 적용

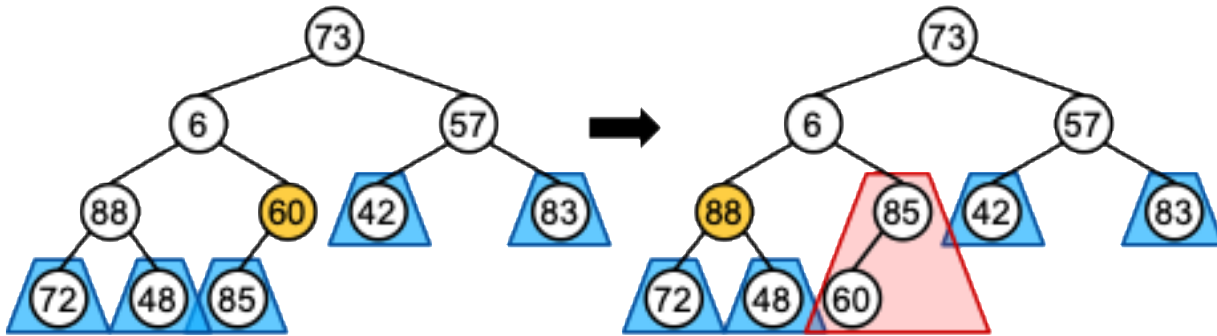


```
def build_heap(arr):  
    n = len(arr)  
    for i in range(n//2-1, -1, -1):  
        siftdown(i, arr, n)
```



BUILD-HEAP

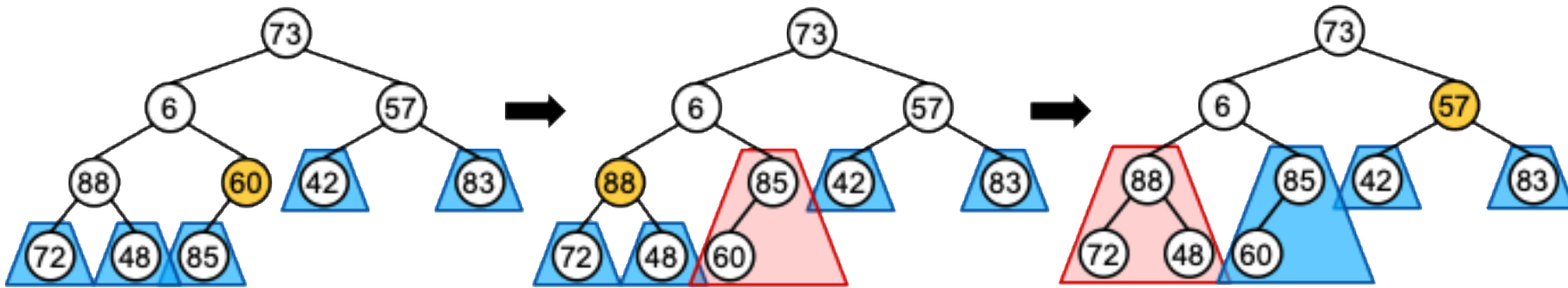
- (자식을 가진) 부모 노드 맨 뒤에서부터 sift down 연산을 반복 적용



```
def build_heap(arr):  
    n = len(arr)  
    for i in range(n//2-1, -1, -1):  
        siftdown(i, arr, n)
```

BUILD-HEAP

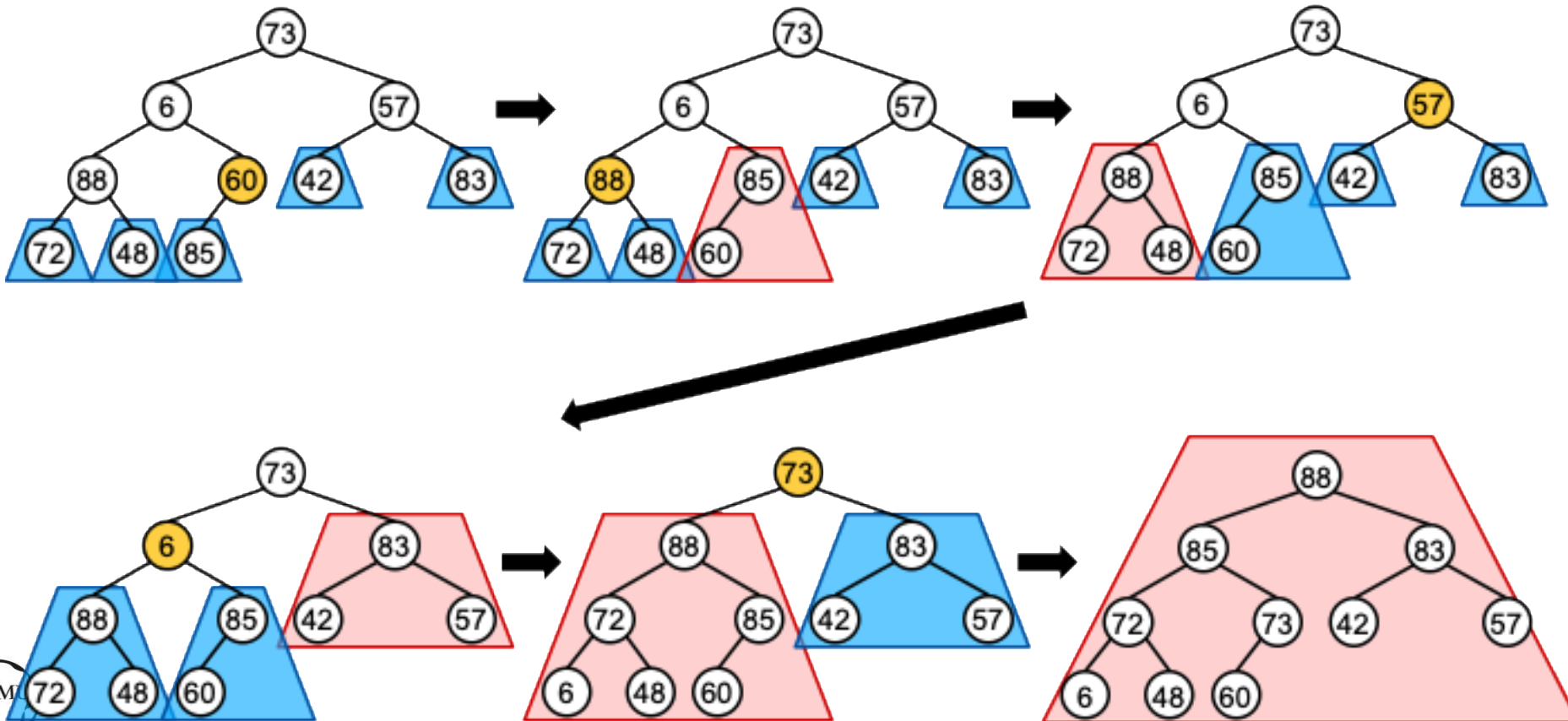
- (자식을 가진) 부모 노드 맨 뒤에서부터 sift down 연산을 반복 적용



```
def build_heap(arr):  
    n = len(arr)  
    for i in range(n//2-1, -1, -1):  
        siftdown(i, arr, n)
```

BUILD-HEAP

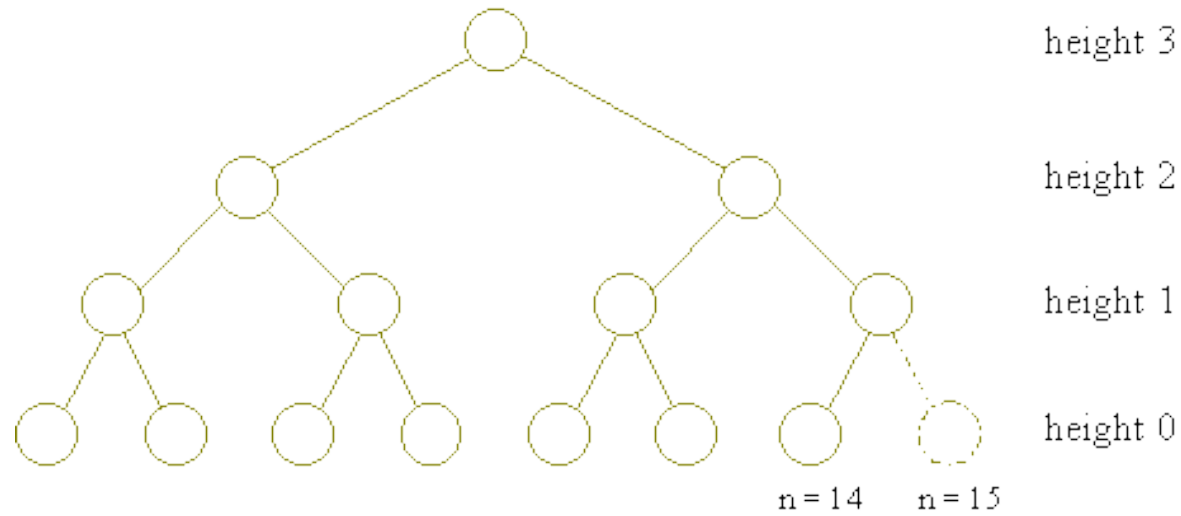
- (자식을 가진) 부모 노드 맨 뒤에서부터 sift down 연산을 반복 적용



BUILD-HEAP

- Complexity analysis (Worst case)

$$\sum_{h=1}^{\lfloor \log n \rfloor} \left\lceil \frac{n}{2^{h+1}} \right\rceil \Theta(h) = \Theta \left(n \sum_{h=1}^{\lfloor \log n \rfloor} \frac{h}{2^h} \right) = \Theta(n)$$



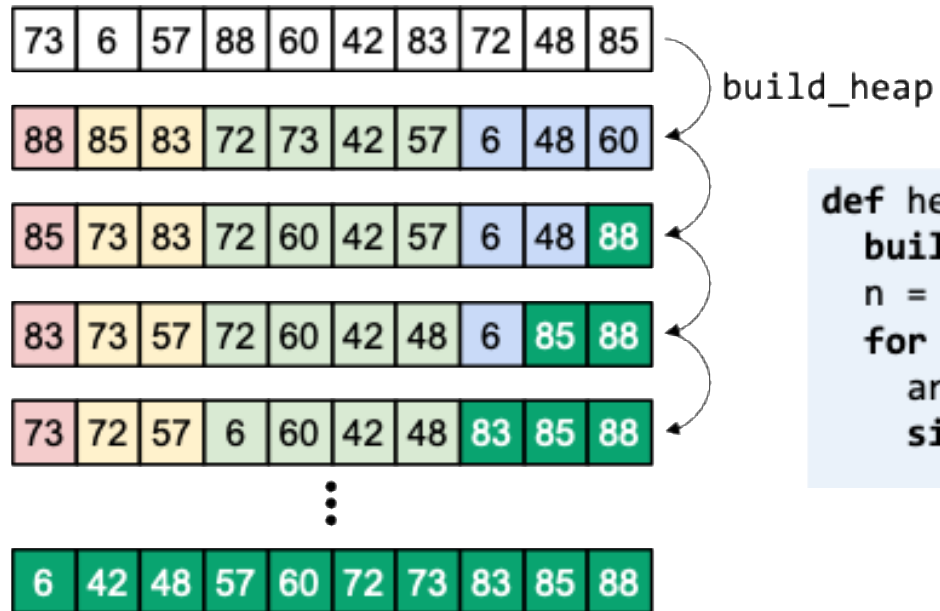


HEAP SORT

The image features a minimalist design on a light gray background. Two thin, dark gray lines intersect: one line slopes downward from left to right, and the other slopes more steeply downward from top-left to bottom-right. To the right of this intersection, the words 'HEAP SORT' are written in a bold, black, sans-serif typeface.

HEAP SORT

- 주어진 배열 (완전 이진 트리)에 대해 Build-heap 수행
- 최댓값 (Root node)을 배열 맨 뒤로 보내고 (Swap) Sift down 작업 반복

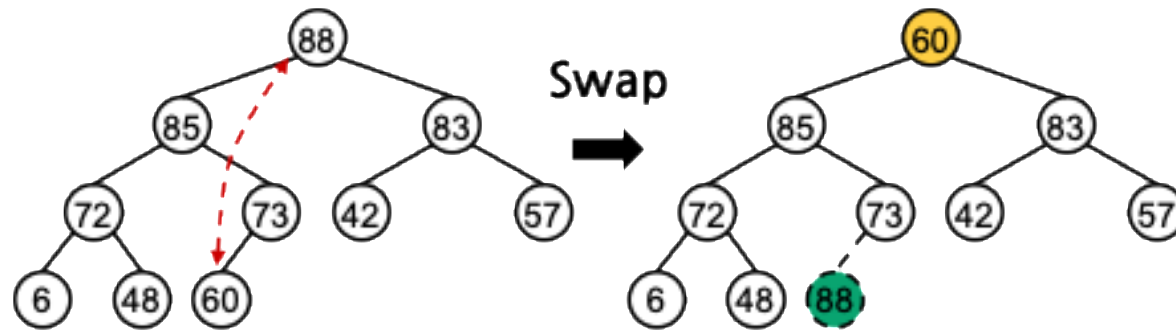


```
def heapsort(arr):  
    build_heap(arr)  
    n = len(arr)  
    for i in range(n-1, 0, -1):  
        arr[0], arr[i] = arr[i], arr[0]  
        siftdown(0, arr, i)
```



HEAP SORT

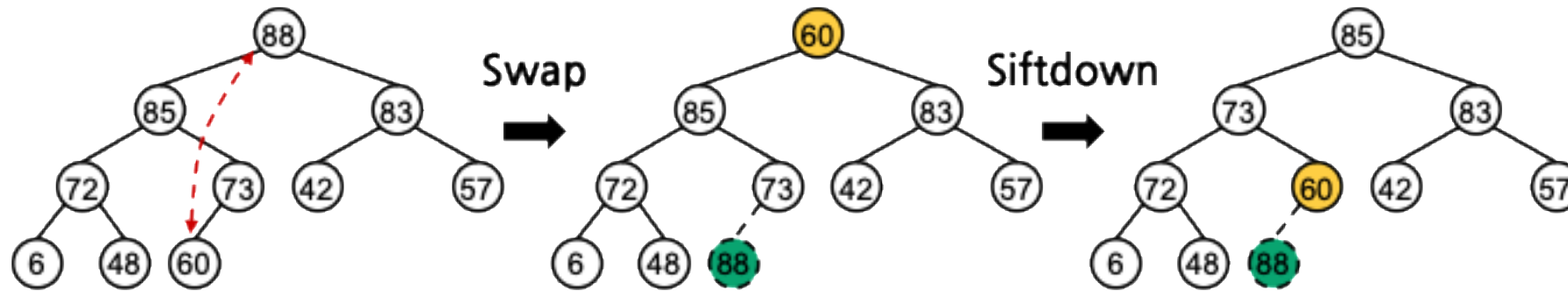
- 최댓값 (Root node)을 배열 맨 뒤로 보내고 (Swap) Sift down 작업 반복



```
...  
for i in range(n-1,0,-1):  
    arr[0], arr[i] = arr[i], arr[0]  
    siftdown(0, arr, i)
```

HEAP SORT

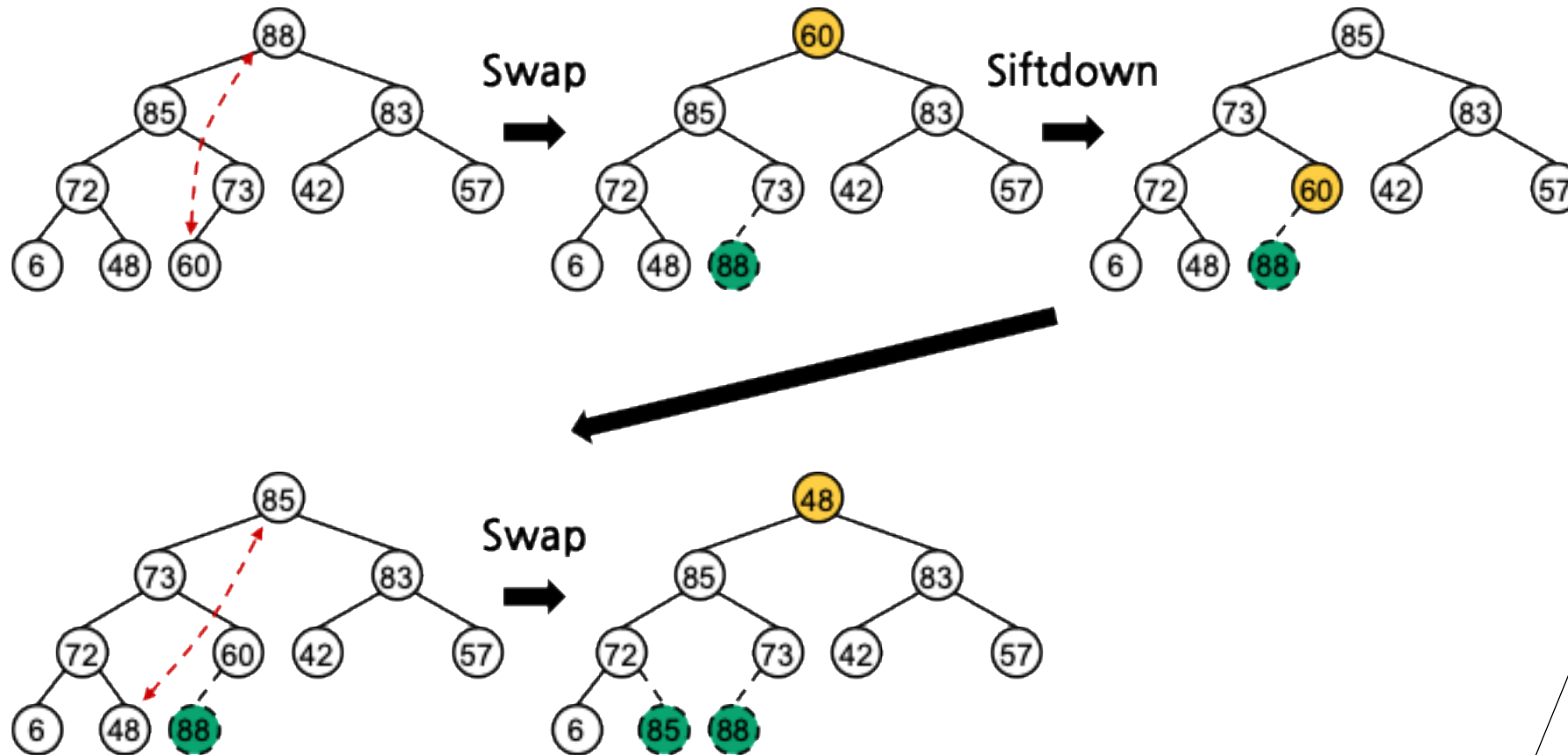
- 최댓값 (Root node)을 배열 맨 뒤로 보내고 (Swap) Sift down 작업 반복



```
...  
for i in range(n-1,0,-1):  
    arr[0], arr[i] = arr[i], arr[0]  
    siftdown(0, arr, i)
```

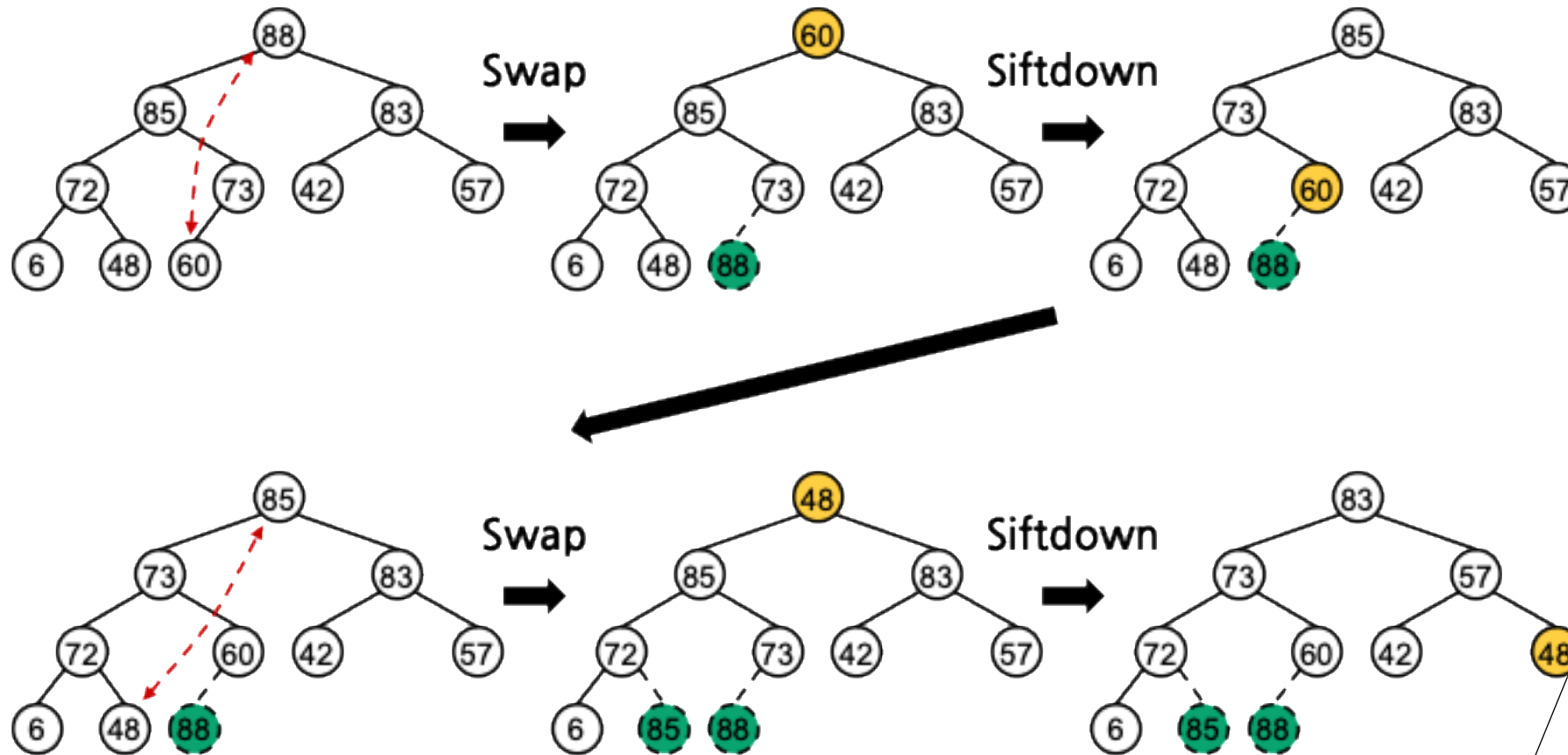
HEAP SORT

- 최댓값 (Root node)을 배열 맨 뒤로 보내고 (Swap) Sift down 작업 반복



HEAP SORT

- 최댓값 (Root node)을 배열 맨 뒤로 보내고 (Swap) Sift down 작업 반복



HEAP SORT

- Complexity analysis

```
def heapsort(arr):  
    build_heap(arr)  
    n = len(arr)  
    for i in range(n-1,0,-1):  
        arr[0], arr[i] = arr[i], arr[0]  
        siftdown(0, arr, i)
```



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They intersect in the upper-left quadrant of the image.

SUMMARY

SUMMARY

1. Heap (data structure)

2. Heapsort



NEXT LECTURE

Sorting (4) – 특수 정렬 알고리즘



Two thin, dark gray lines intersect diagonally on a light gray background. One line runs from the top-left towards the bottom-right, and the other runs from the top-right towards the bottom-left. They cross each other in the upper-left quadrant of the image.

QUESTIONS?